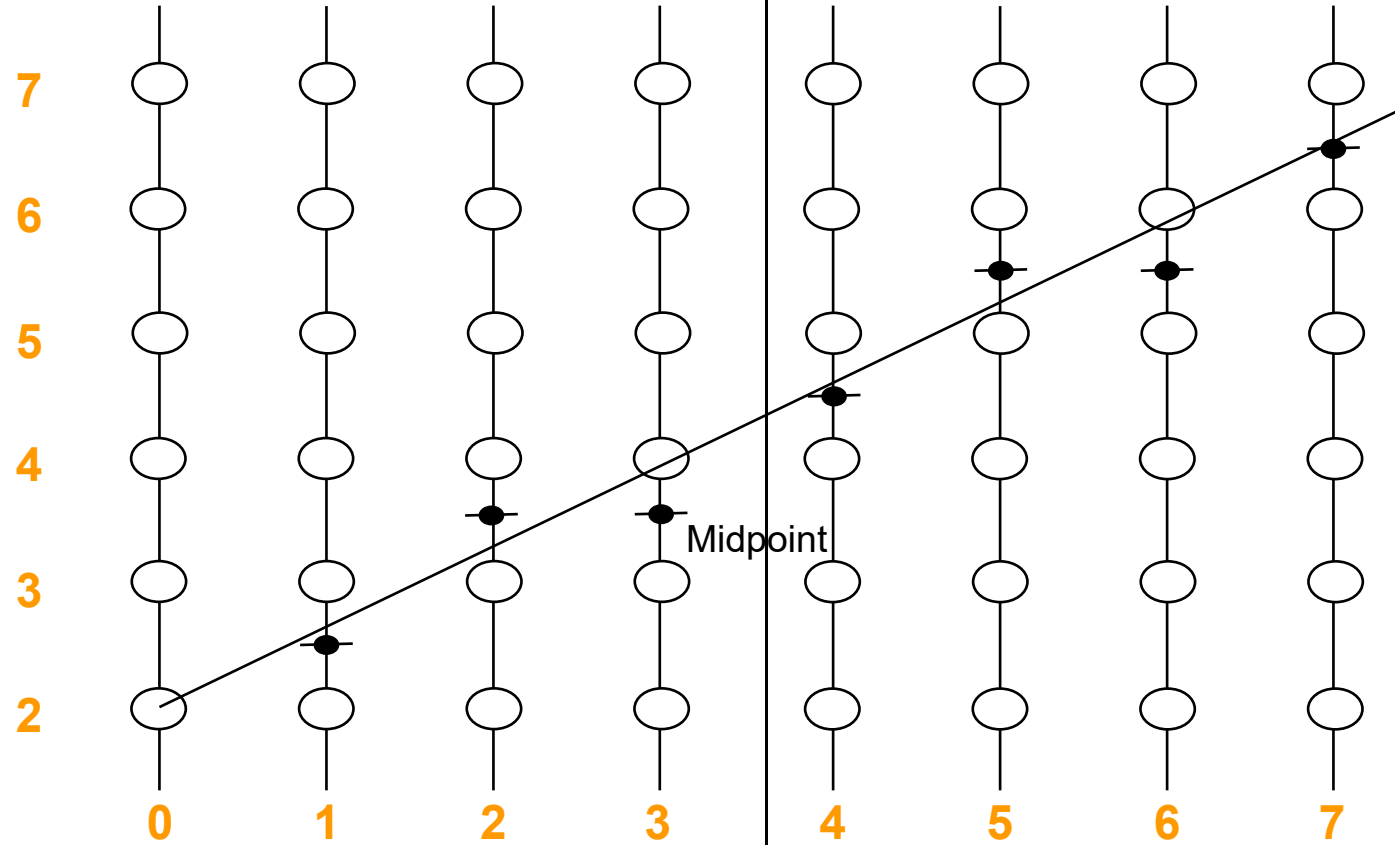
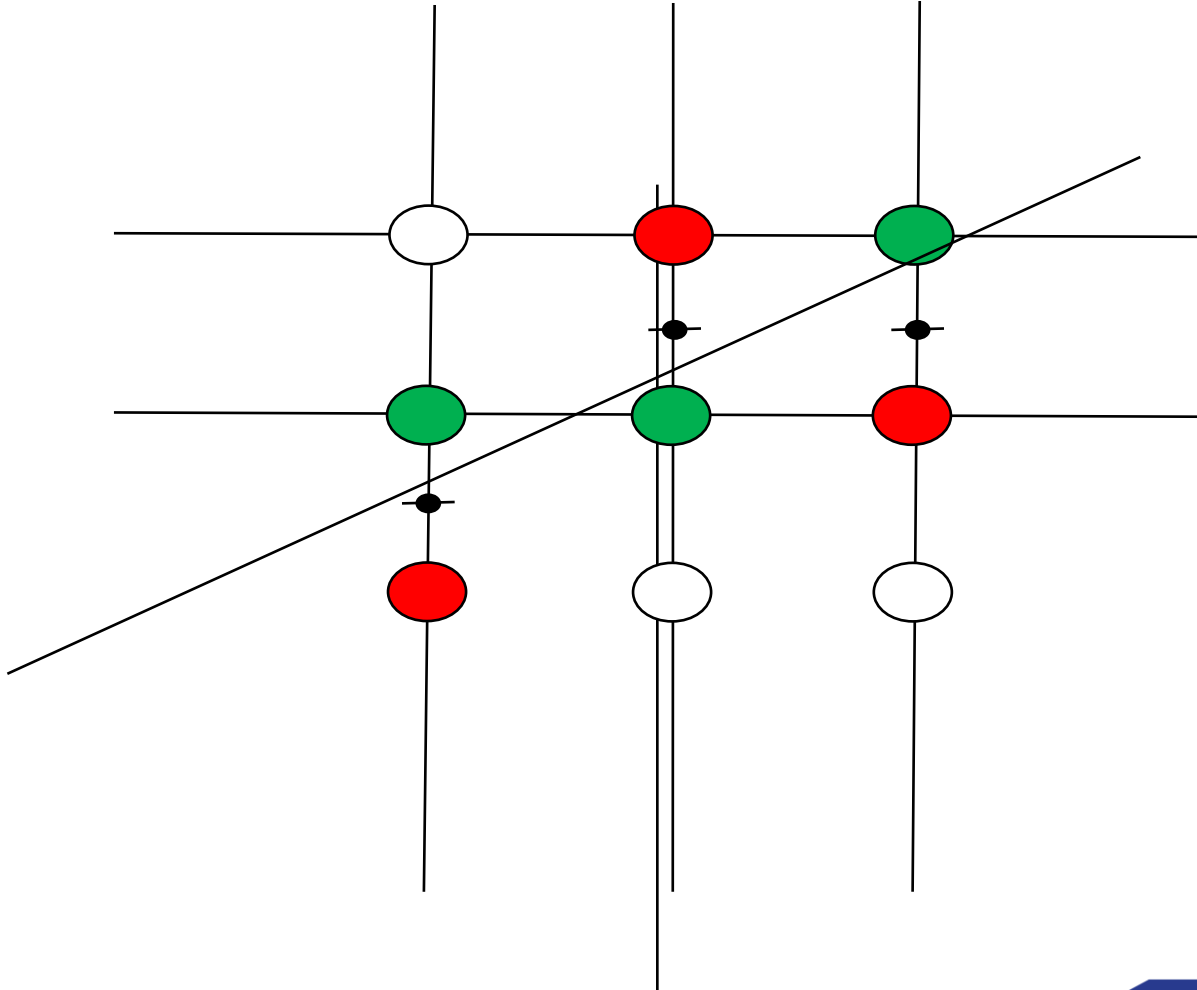
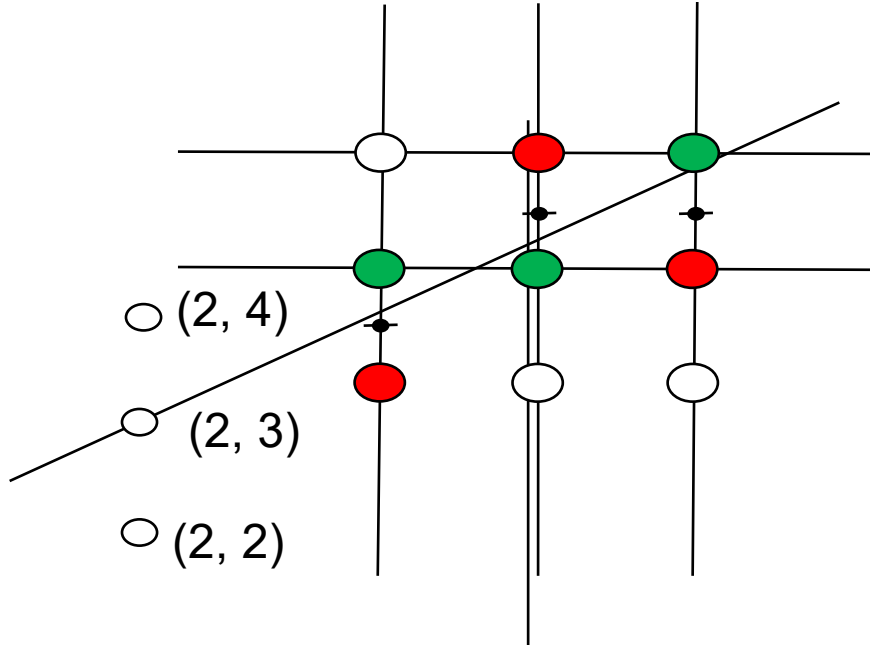


Computer Graphics: Line Drawing Algorithms

Scan Conversion Algorithms
(Midpoint Line)





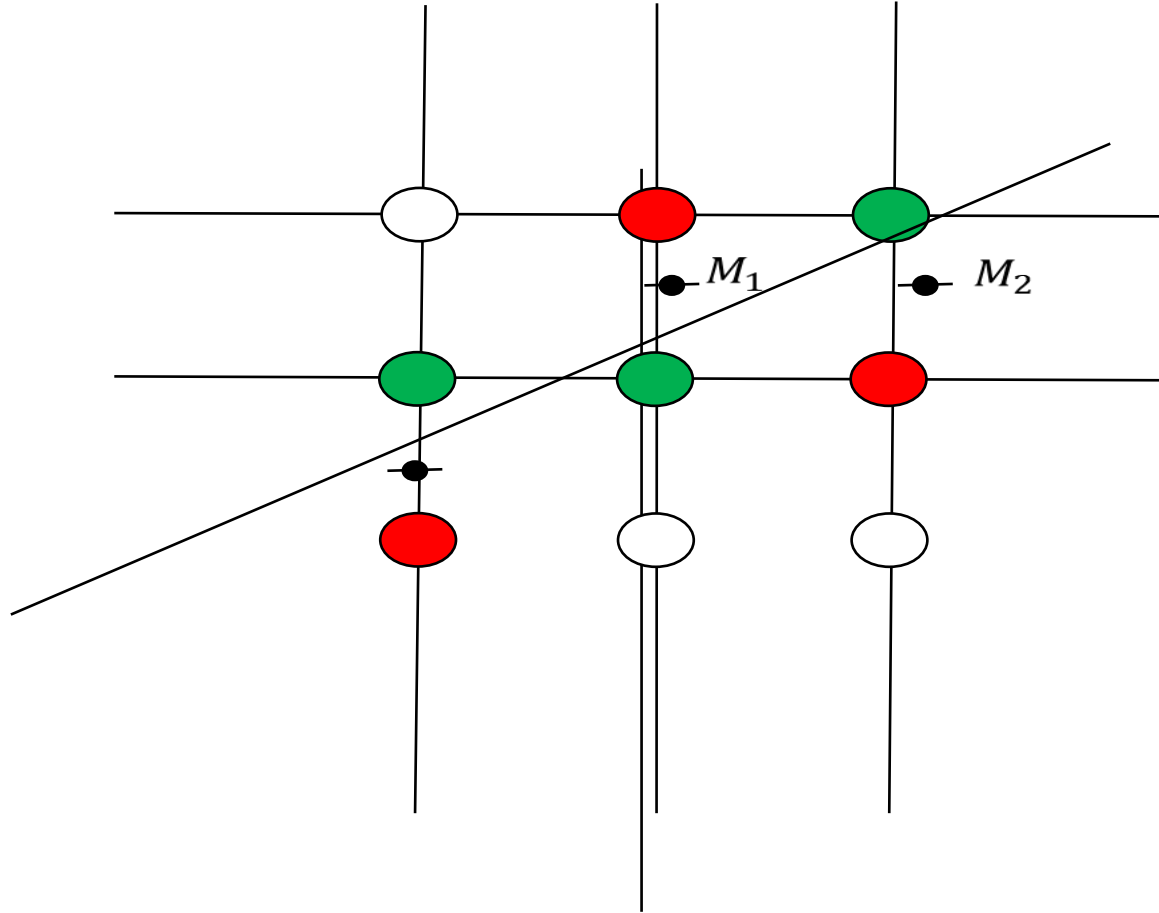


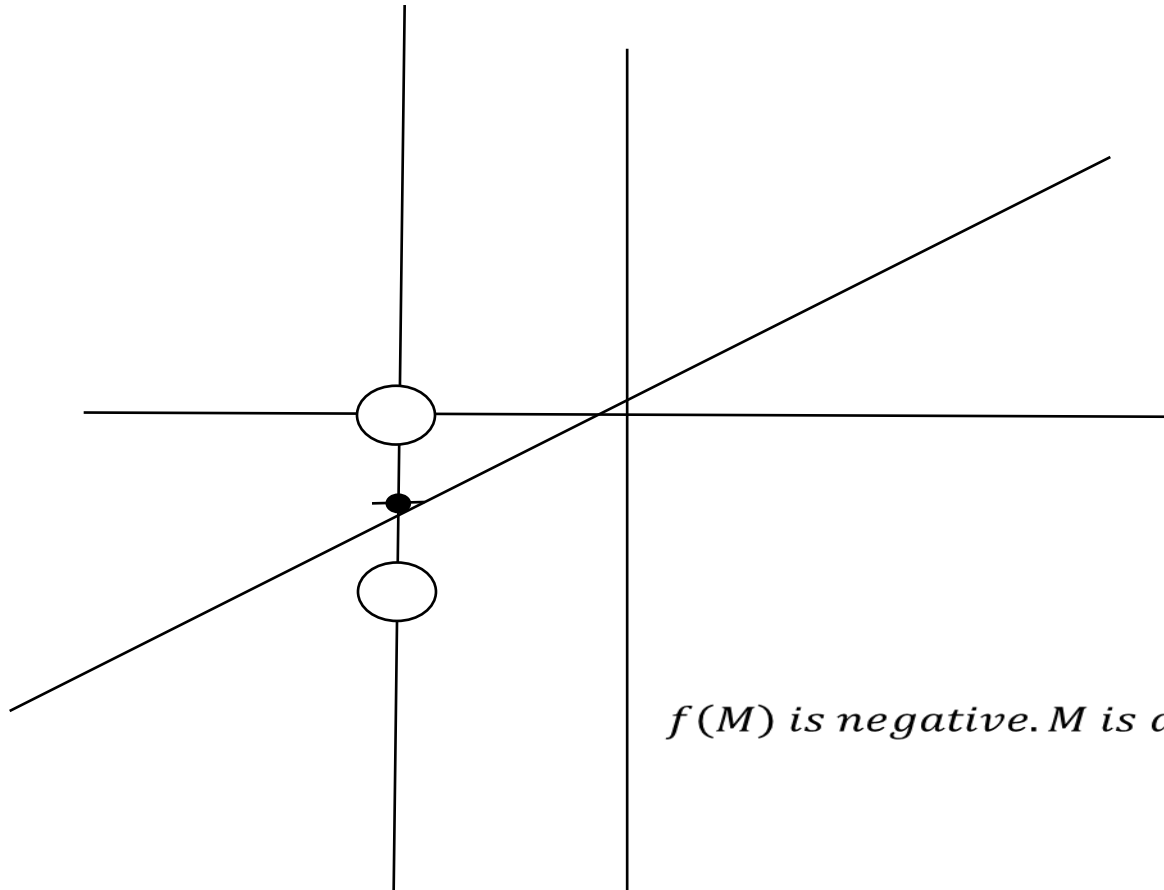
$$ax + by + c = 0$$

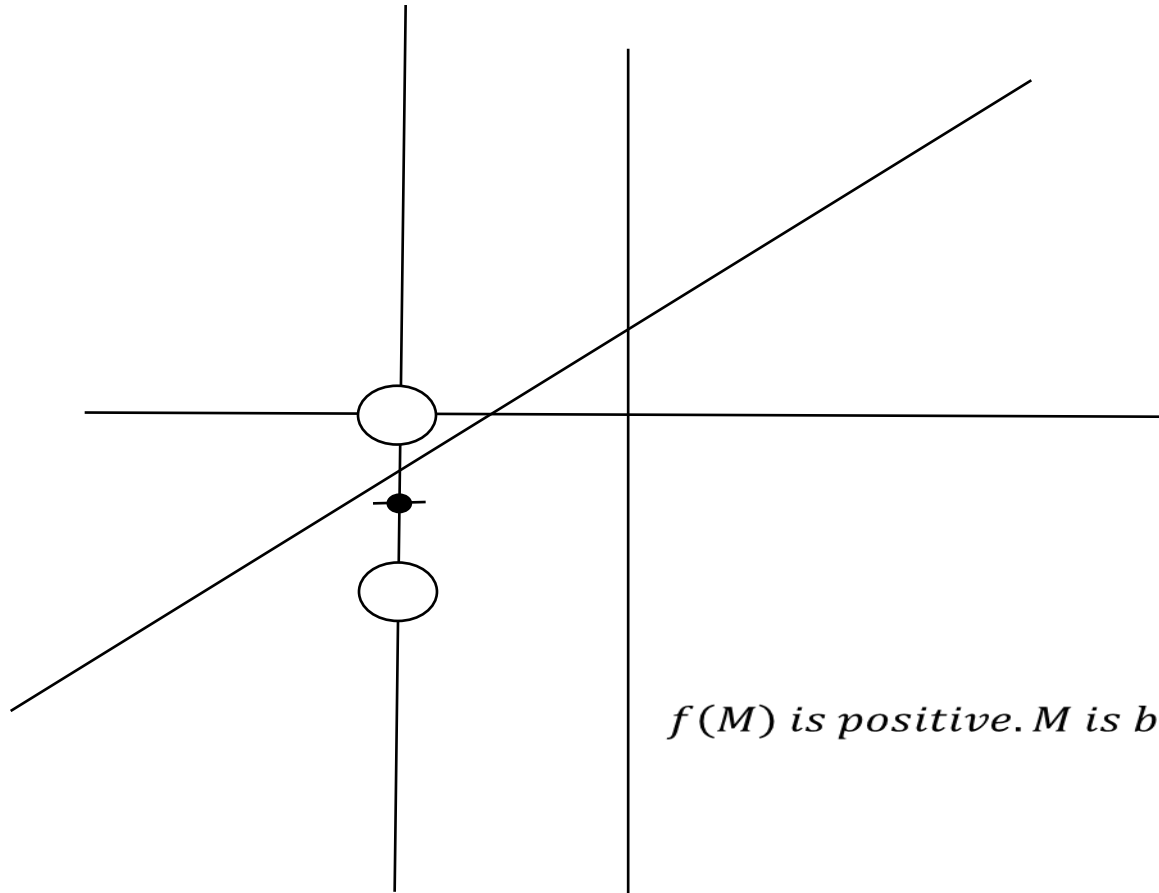
$$f(x, y) = ax + by + c$$

$$f(2, 4) = (-)ve$$

$$f(2, 2) = (+)ve$$

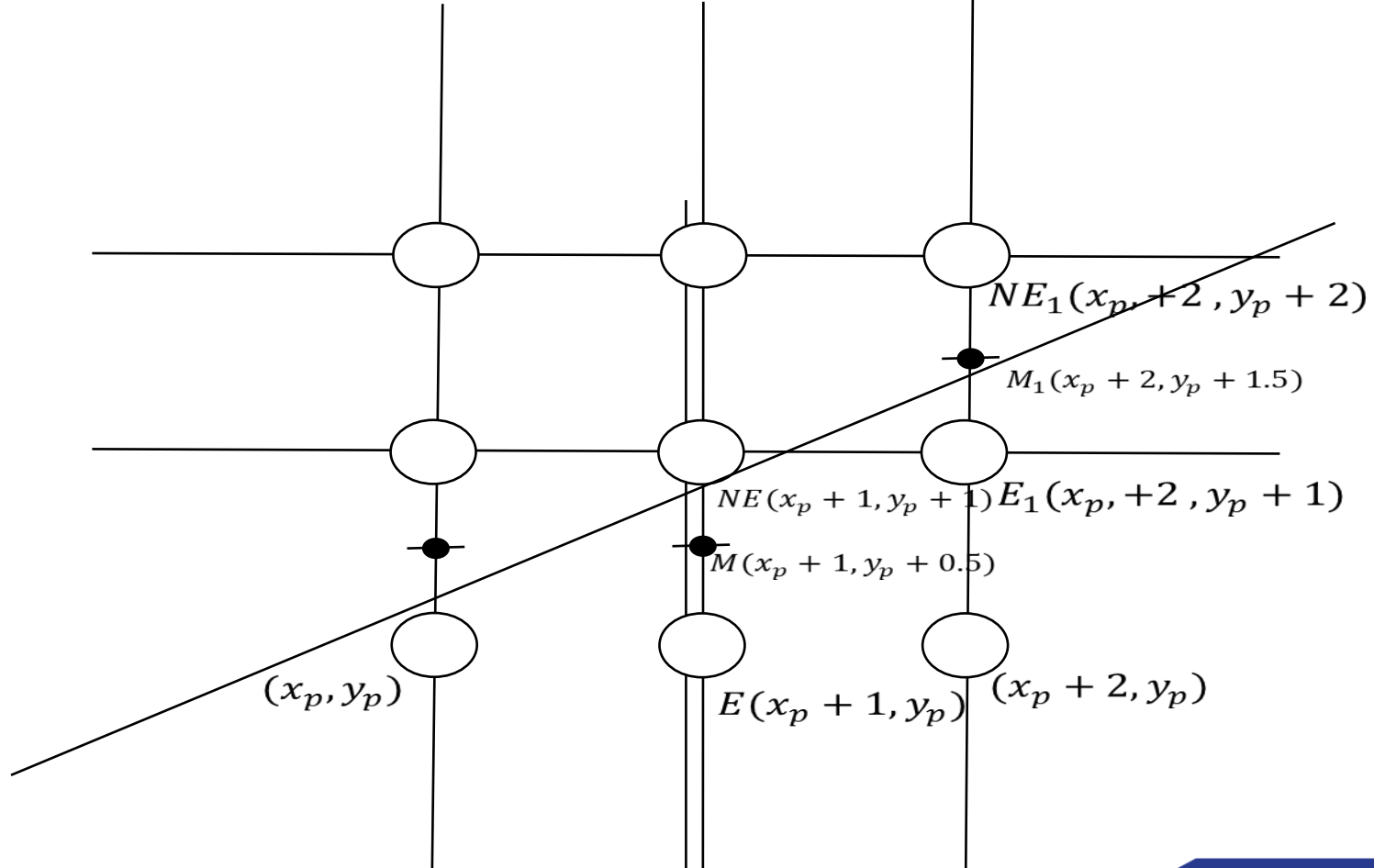


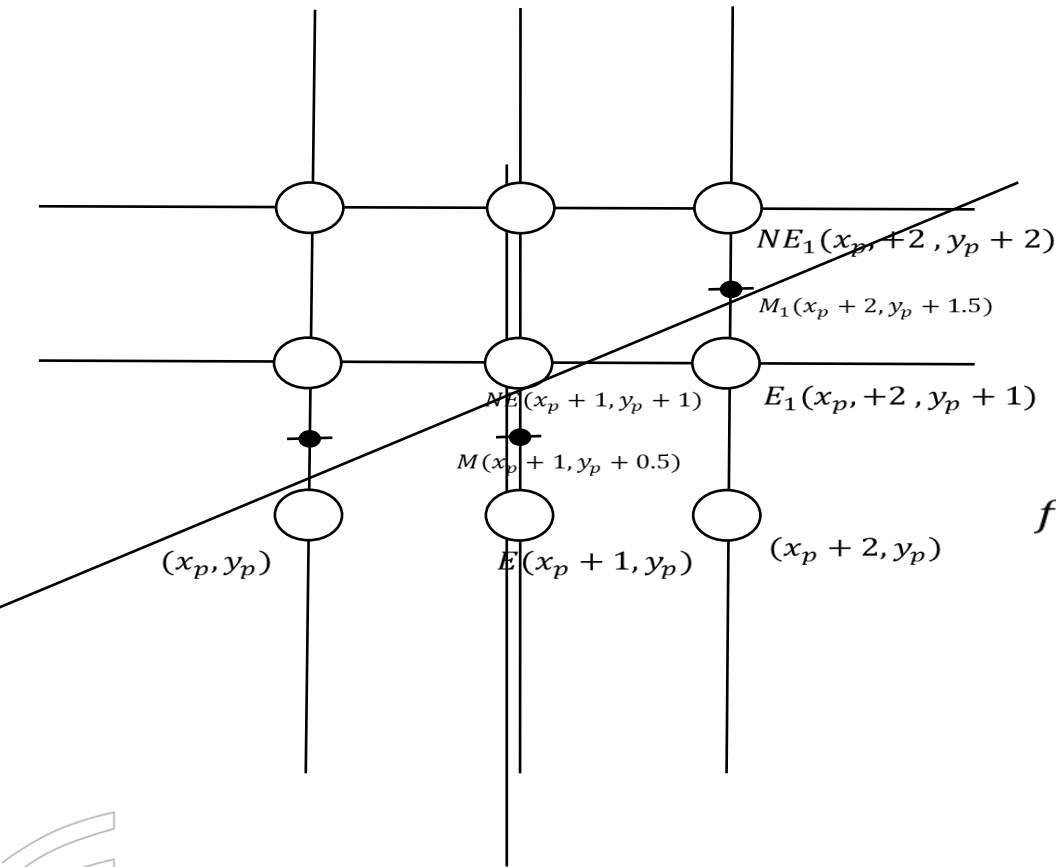




$f(M)$	Pixel chosen
$f(M) > 0$	upper (NE)
$f(M) \leq 0$	lower (E)

Derivation of MPL Algorithm for Zone - 0



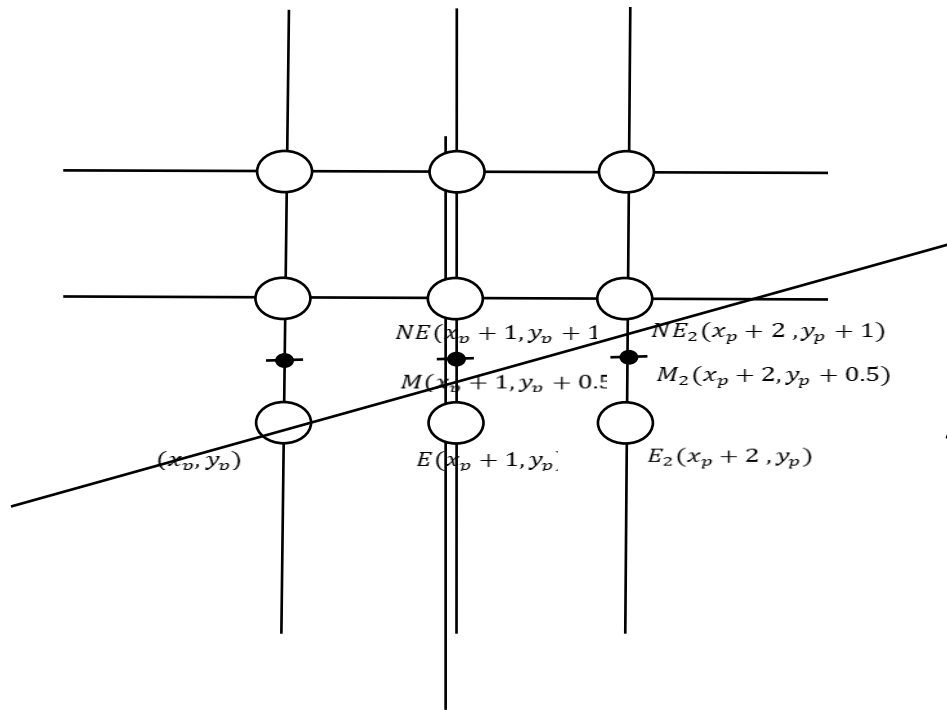


$$\begin{aligned}
 f(M) &= f(x_p + 1, y_p + 0.5) \\
 &= a(x_p + 1) + b(y_p + 0.5) + c \\
 d &= ax_p + by_p + c + a + 0.5b
 \end{aligned}$$

Assume, $d > 0$, so NE is chosen

$$\begin{aligned}
 f(M_1) &= f(x_p + 2, y_p + 1.5) \\
 &= a(x_p + 2) + b(y_p + 1.5) + c \\
 d_{new} &= ax_p + by_p + c + 2a + 1.5b
 \end{aligned}$$

$$\begin{aligned}
 d_{new} - d &= a + b \\
 d_{new} &= d + a + b
 \end{aligned}$$



$$\begin{aligned}
 f(M) &= f(x_p + 1, y_p + 0.5) \\
 &= a(x_p + 1) + b(y_p + 0.5) + c \\
 d &= ax_p + by_p + c + a + 0.5b
 \end{aligned}$$

Again assume, $d \leq 0$, so E is chosen

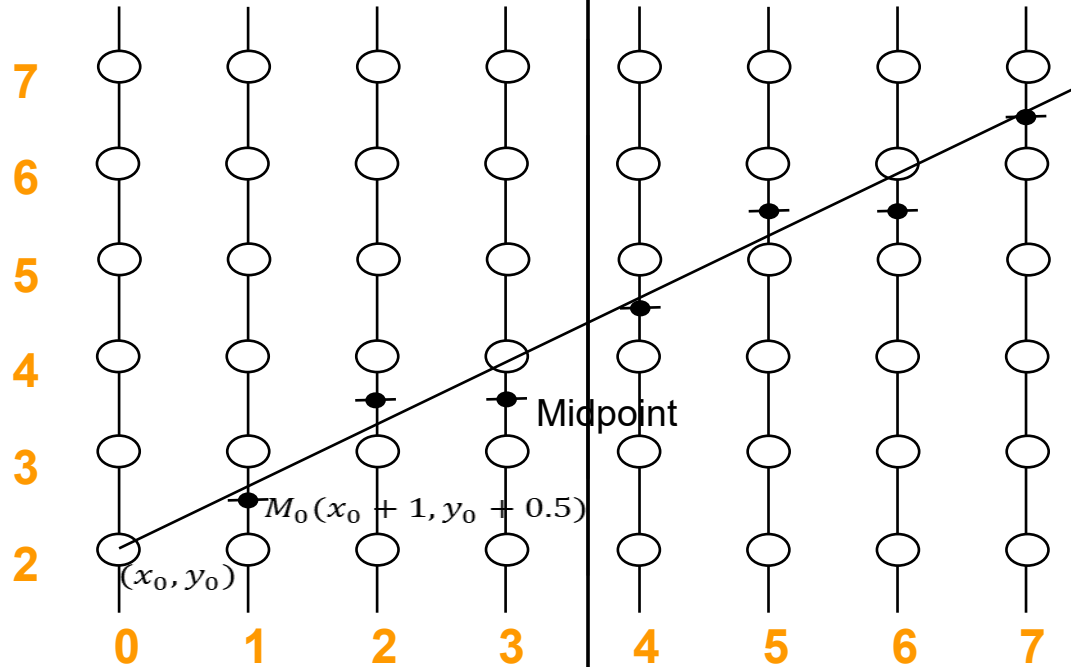
$$\begin{aligned}
 f(M_2) &= f(x_p + 2, y_p + 0.5) \\
 &= a(x_p + 2) + b(y_p + 0.5) + c \\
 d_{new} &= ax_p + by_p + c + 2a + 0.5b \\
 d_{new} - d &= a \\
 d_{new} &= d + a
 \end{aligned}$$

Calculate d for 1st column.

Choose E/NE.

Update d_{new} acc. to E/NE.

Use d_{new} to choose E/NE again and repeat the loop until the end.

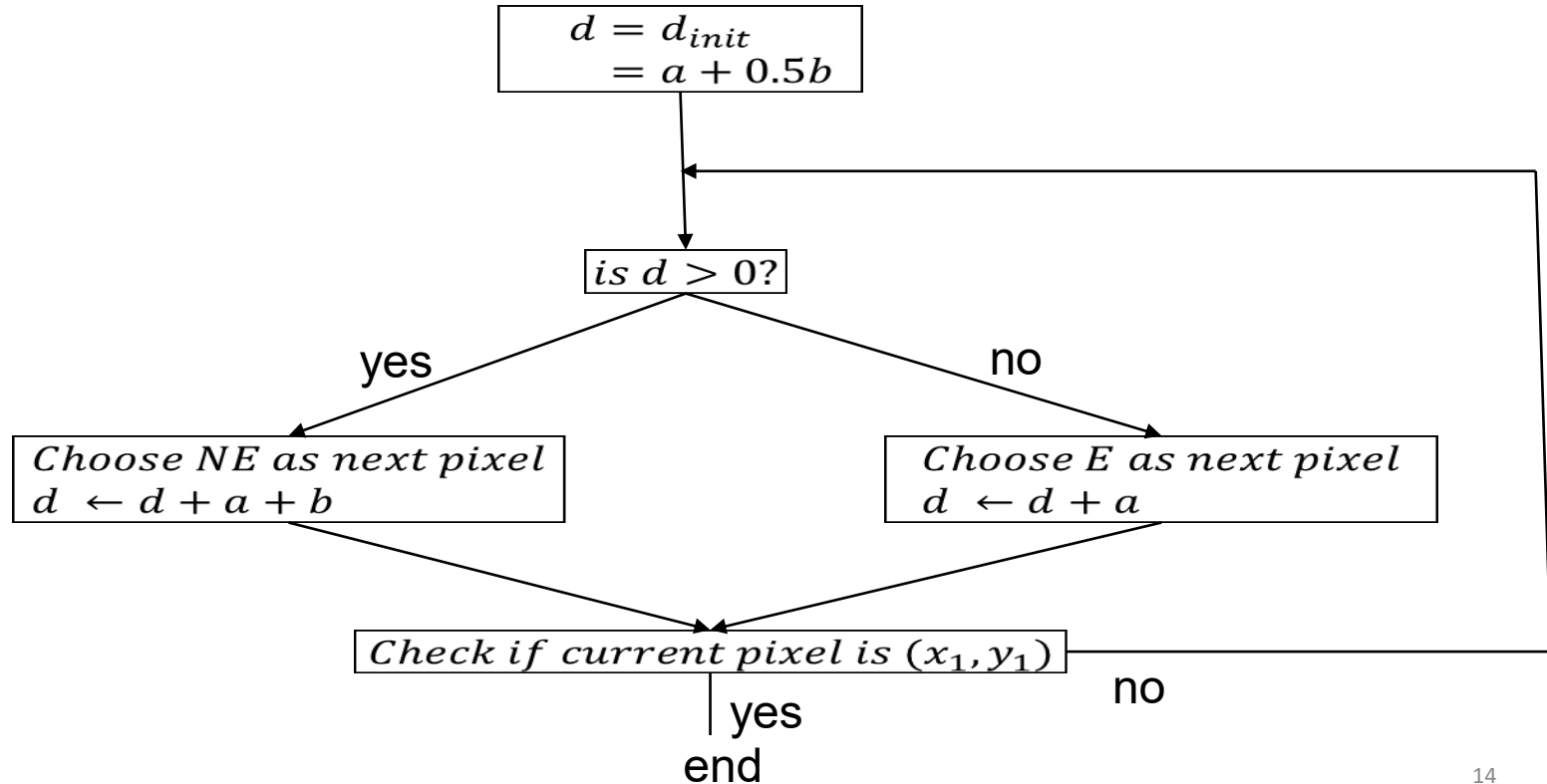


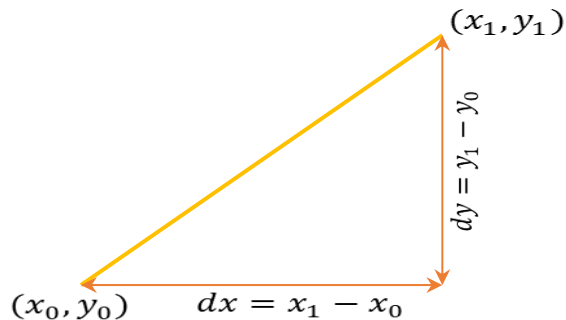
$$\begin{aligned}
 d_{init} &= f(M_0) \\
 &= f(x_0 + 1, y_0 + 0.5) \\
 &= a(x_0 + 1) + b(y_0 + 0.5) + c \\
 &= ax_0 + a + by_0 + 0.5b + c \\
 d_{init} &= ax_0 + by_0 + c + a + 0.5b
 \end{aligned}$$

$$\begin{aligned}
 ax + by + c &= 0 \\
 \text{true for all } (x, y) \text{ on the line}
 \end{aligned}$$

$$\text{So, } ax_0 + by_0 + c = 0$$

$$\begin{aligned}
 d_{init} &= ax_0 + by_0 + c + a + 0.5b \\
 &= 0 + a + 0.5b \\
 &= a + 0.5b
 \end{aligned}$$





$$y = mx + B$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx} \cdot x + B$$

$$y \cdot dx = dy \cdot x + B \cdot dx$$

$$0 = dy \cdot x - y \cdot dx + B \cdot dx$$

$$dy \cdot x - dx \cdot y + B \cdot dx = 0$$

Comparing this with,

$$ax + by + c = 0$$

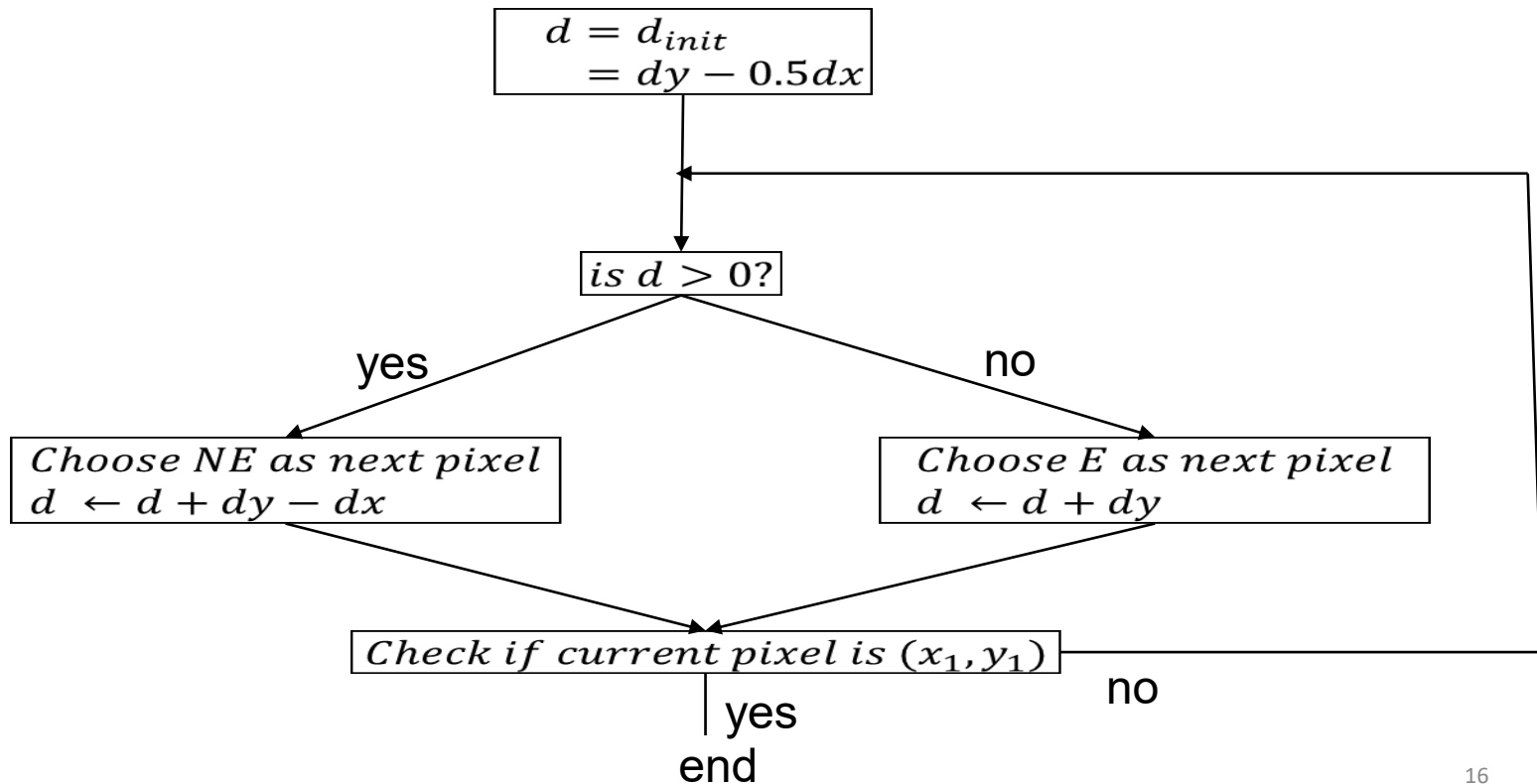
We get,

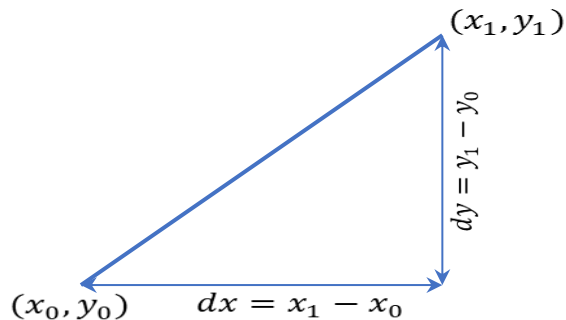
$$a = dy$$

$$b = -dx$$

$$c = B \cdot dx$$

Putting,
 $a = dy$
 $b = -dx$





$$y = mx + b$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx} \cdot x + b$$

$$y \cdot dx = dy \cdot x + b \cdot dx$$

$$0 = dy \cdot x - y \cdot dx + b \cdot dx$$

$$\mathbf{dy \cdot x - dx \cdot y + b \cdot dx = 0}$$

Comparing this with,

$$\mathbf{ax + by + c = 0}$$

We get,

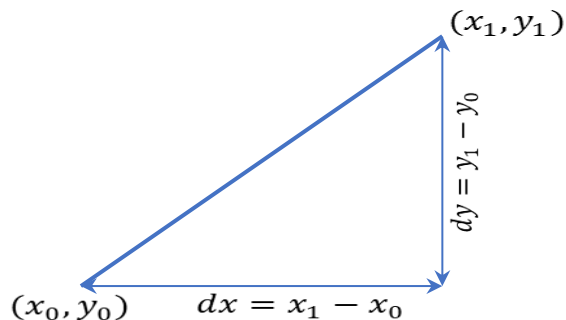
$$a = dy$$

$$b = -dx$$

$$c = b \cdot dx$$

$$2x + 3y + 1 = 0$$

$$4x + 6y + 2 = 0$$



$$y = mx + b$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx} \cdot x + b$$

$$y \cdot dx = dy \cdot x + b \cdot dx$$

$$0 = dy \cdot x - y \cdot dx + b \cdot dx$$

$$2dy \cdot x - 2dx \cdot y + 2b \cdot dx = 0$$

Comparing this with,

$$ax^2 + by^2 + c = 0$$

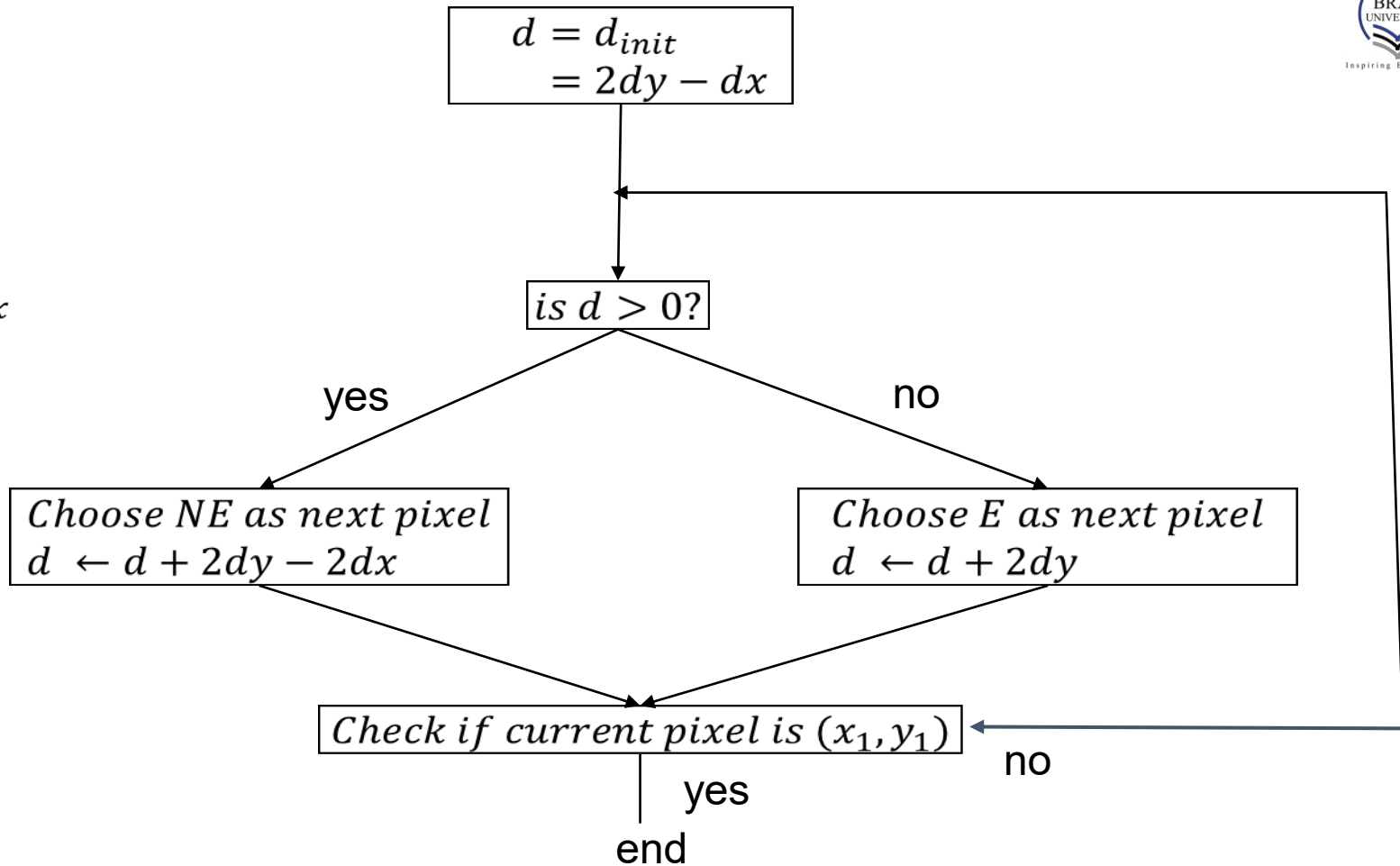
We get,

$$a = 2dy$$

$$b = -2dx$$

$$c = 2b \cdot dx$$

Putting,
 $a = 2dy$
 $b = -2dx$



MPL Algorithm:

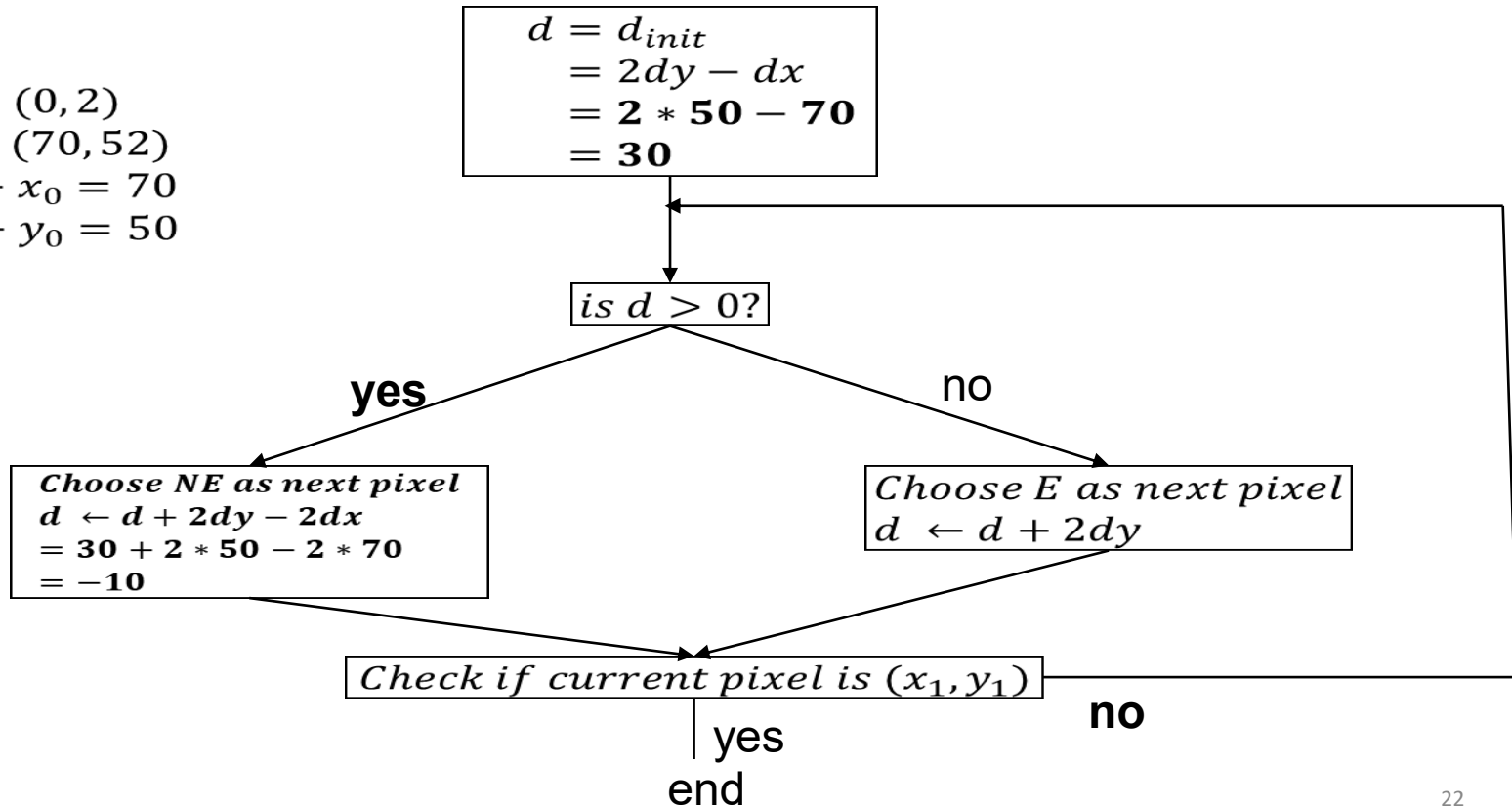
```

func MidpointLine(int x0, int y0, int x1, int y1, int value){
    int dx, dy, incrE, incrNE, d, x, y;
    dx = x1 -x0;
    dy = y1 -y0;
    d = 2 * dy - dx;
    incrE = 2 * dy;
    incrNE = 2 * (dy - dx);
    x = x0;
    y = y0;
    WritePixel (x, y, value);
    while (x < x1) {
        if (d <= 0) {
            //choose E
            d = d + incrE;
            x = x + 1;
        }
        else {
            //choose NE
            d = d + incrNE;
            x = x + 1;
            y = y + 1;
        }
        WritePixel (x,y, value) //The selected pixel closest to the line
    }
}
  
```

Simulation Problem

Find out the **first 7 pixels** of the line segment starting from $(0, 2)$ to $(70, 52)$ using midpoint line algorithm

$(x_0, y_0) = (0, 2)$
 $(x_1, y_1) = (70, 52)$
 $dx = x_1 - x_0 = 70$
 $dy = y_1 - y_0 = 50$



x	y	d	NE(+1,+1)/E(+1,0)	d updating	Pixel
0	2	30	NE	$30+2 \times 50-2 \times 70 = -10$	(0, 2)
1	3				

$$d_{init} = 2dy - dx = 2.50 - 70 = 30$$

$$\Delta d_{NE} = 2dy - 2dx = 2.50 - 2.70 = -40$$

$$\Delta d_E = 2dy = 2.50 = 100$$

			Decision for Next Pixel		
x	y	d	NE(+1,+1)/ E(+1,0)	d updating	Pixel
0	2	30	NE	$30 + \Delta d_{NE} = -10$	(0, 2)
1	3	-10	E	$-10 + \Delta d_E = 90$	(1, 3)
2	3	90	NE	$90 + \Delta d_{NE} = 50$	(2, 3)
3	4	50	NE	$50 + \Delta d_{NE} = 10$	(3, 4)
4	5	10	NE	$10 + \Delta d_{NE} = -30$	(4, 5)
5	6	-30	E	$-30 + \Delta d_E = 70$	(5, 6)
6	6	70	NE	$70 + \Delta d_{NE} = 30$	(6, 6)
7	7				

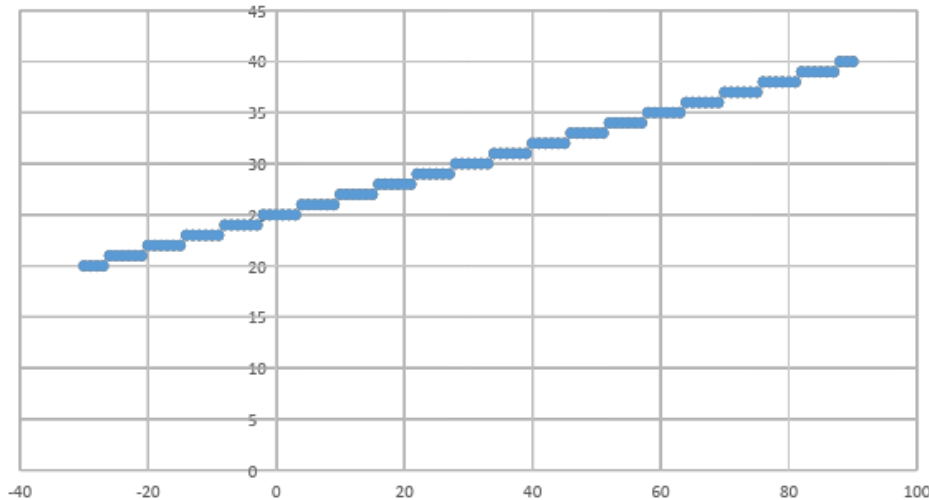
Midpoint Line Drawing Algorithm

```
Midpoint ( $x_1, y_1, x_2, y_2$ ) {  
     $dx = x_2 - x_1$  ;  $dy = y_2 - y_1$  ;  
     $D = 2*dy - dx$  ;  $\Delta NE = 2*(dy-dx)$  ;  $\Delta E = 2*dy$  ;  
     $x = x_1$  ;  $y = y_1$  ;  
    while(  $x \leq x_2$  ) {  
        Draw( $x, y$ ) ;  
         $x++$  ;  
        if ( $D > 0$ ) {  
             $y++$  ;  
             $D = D + \Delta NE$  ;  
        }  
        else {  
             $D = D + \Delta E$  ;  
        }  
    }  
}
```

$(-30, 20)$ to $(90, 40)$

$$dx = 90 + 30 = 120 ; dy = 40 - 20 = 20;$$

$$D = 2*20 - 120 = -80; \Delta NE = 2*(20-120) = -200 ; \Delta E = 2*20 = 40;$$



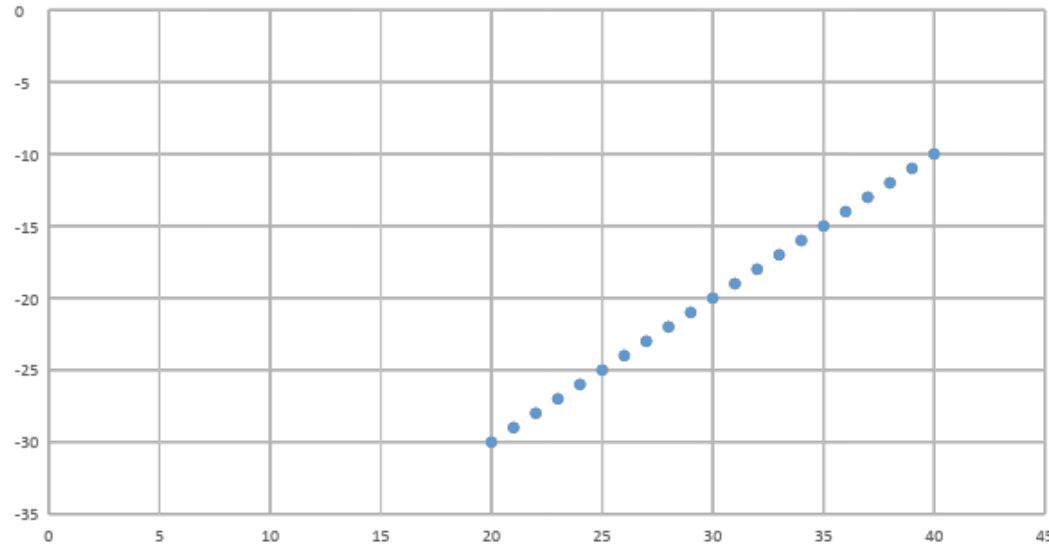
$$m = \frac{20}{120} = 0.167 < 1$$

X	Y	D
-30	20	-80
-29	20	-40
-28	20	0
-27	20	40
-26	21	-160
-25	21	-120
-24	21	-80
-23	21	-40
-22	21	0
-21	21	40
-20	22	-160
-19	22	-120
-18	22	-80
-17	22	-40
-16	22	0
-15	22	40
-14	23	-160
-13	23	-120
-12	23	-80
-11	23	-40

(20, -30) to (40, 90)

$$dy = 90 + 30 = 120 ; dx = 40 - 20 = 20 ;$$

$$D = 2 * 120 - 20 = 220 ; \Delta NE = 2 * (120 - 20) = 200 ; \Delta E = 2 * 120 = 240$$



$$m = \frac{120}{20} = 6 > 1$$

X	Y	D
20	-30	220
21	-29	420
22	-28	620
23	-27	820
24	-26	1020
25	-25	1220
26	-24	1420
27	-23	1620
28	-22	1820
29	-21	2020
30	-20	2220
31	-19	2420
32	-18	2620
33	-17	2820
34	-16	3020
35	-15	3220
36	-14	3420
37	-13	3620
38	-12	3820

(30, -20) to (-90, 40)

-
- If we start from (30, -20), then we need to decrement x to reach (-90, 40)
- If we start from (-90, 40), x will be incremented to reach (30, -20) but y needs to be decremented!
- $m = \frac{60}{-120} = -0.5 < 0$

Midpoint (x_1, y_1, x_2, y_2) {

$dx = x_2 - x_1$; $dy = y_2 - y_1$;

$D = 2 * \underline{dy} - dx$; $\Delta NE = 2 * (\underline{dy} - dx)$; $\Delta E = 2 * \underline{dy}$;

$x = x_1$; $y = y_1$;

while($x \leq x_2$) {

Draw(x, y);

$x++$;

if ($D > 0$) {

$y++$;

$D = D + \Delta NE$;

}

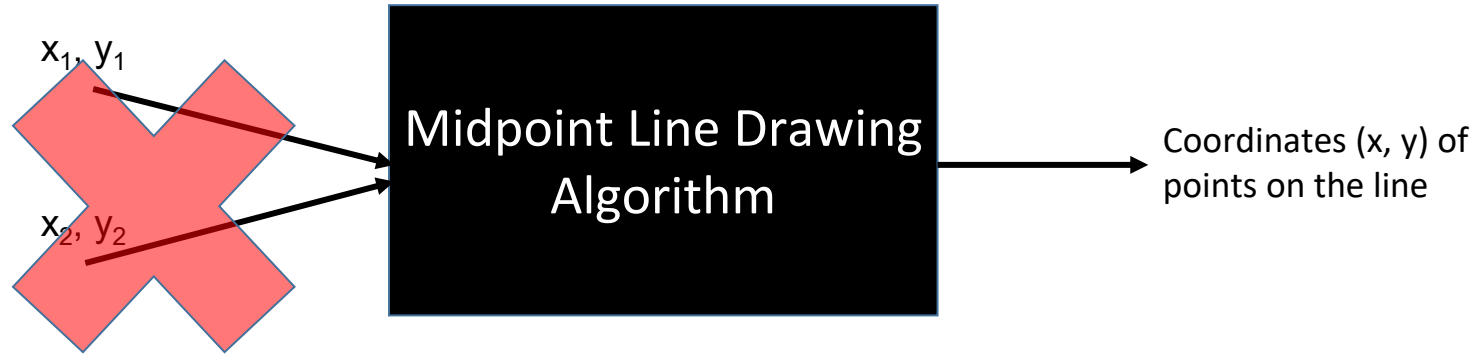
else {

$D = D + \Delta E$;

}

}

}

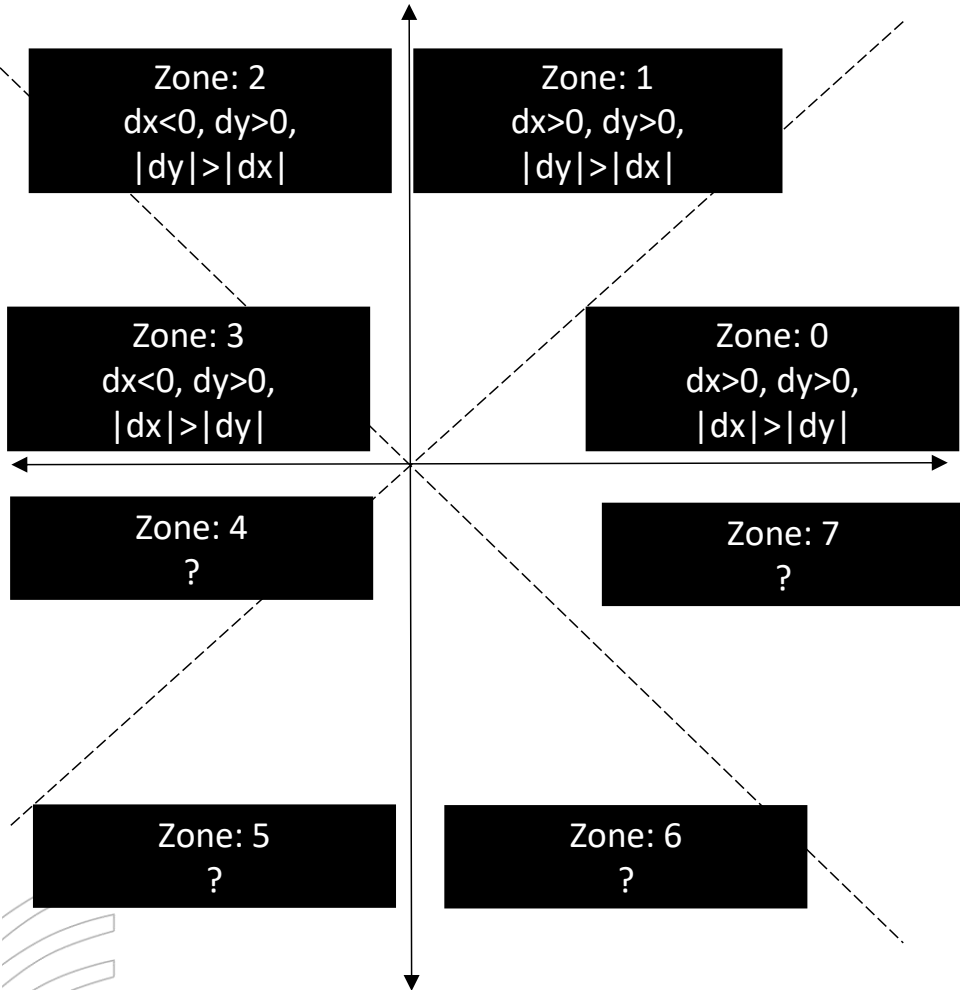


Eight-way Symmetry

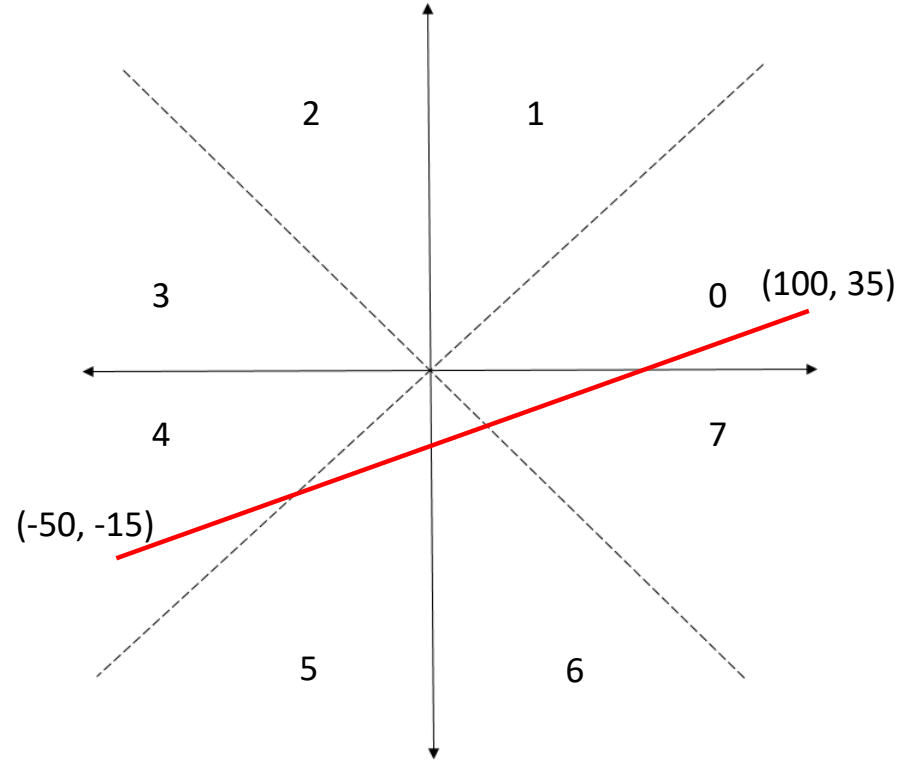
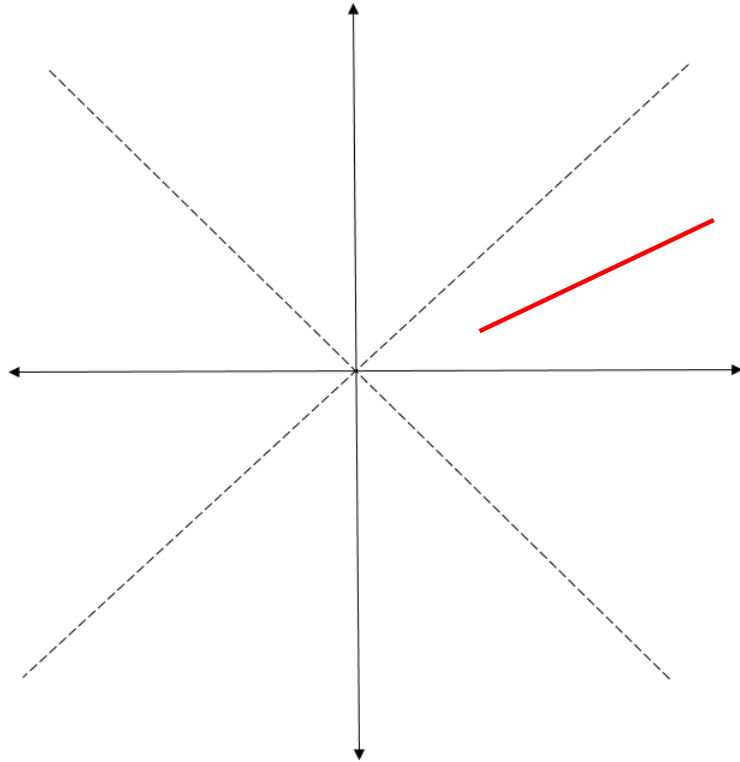
Check this article to understand 8-way symmetry:

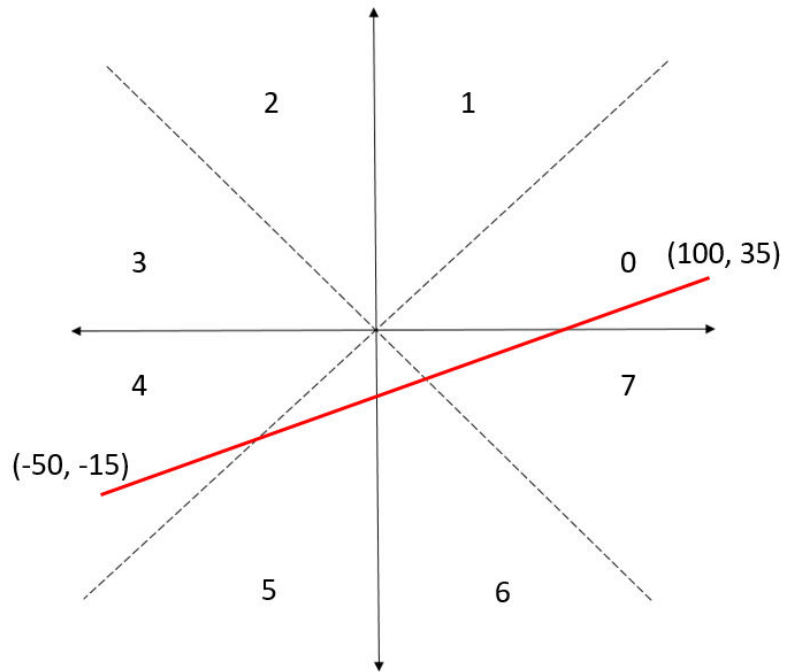
<https://studyresearch.in/bresenhams-circle-algorithm/>

How to find Zone?



```
FindZone( $x_1, y_1, x_2, y_2$ ){  
     $dx = x_2 - x_1$  ;  $dy = y_2 - y_1$  ;  
  
    if( $|dx| > |dy|$ ){  
        if( $dx > 0 \ \&\& \ dy > 0$ ) zone = 0;  
        else if( $dx < 0 \ \&\& \ dy > 0$ ) zone = 3;  
        else if ( ? ? ) zone = ? ;  
        else if ( ? ? ) zone = ?  
    }  
  
    else{  
        if( $dx > 0 \ \&\& \ dy > 0$ ) zone = 1;  
        else if( $dx < 0 \ \&\& \ dy > 0$ ) zone = 2;  
        else if ( ? ? ) zone = ? ;  
        else if ( ? ? ) zone = ?  
    }  
}
```





$$dx = 100 + 50 = 150 > 0$$

$$dy = 35 + 15 = 50 > 0$$

$$|dx| > |dy|$$

$$\text{Zone} = 0$$

How do we utilize the zones?

```

Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy - dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
    
```

Input (x₁, y₁) to (x₂, y₂) for a line of Zone M, where M={0, 1, ..., 7}

FindZone

Convert the coordinates of a line in Zone M into the coordinates of a line in Zone 0

?

Use the existing midpoint line drawing algorithm for Zone 0

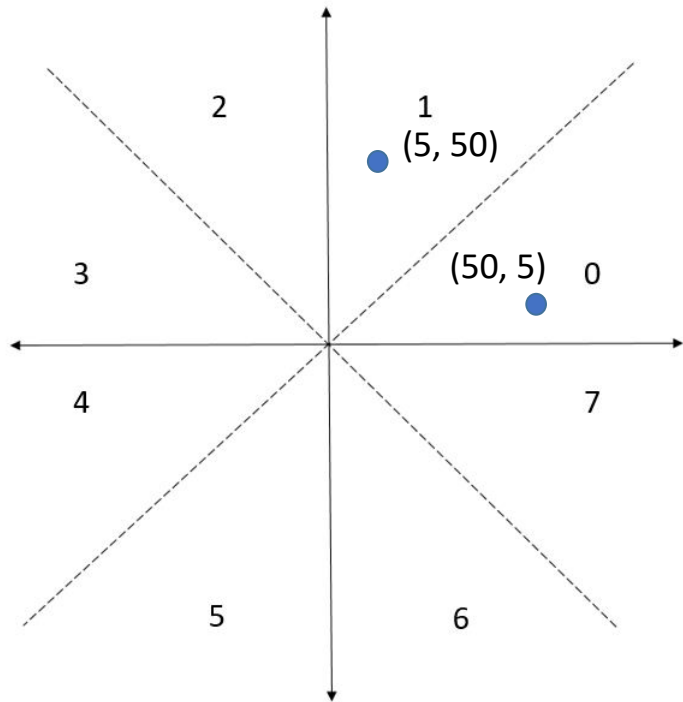
MidPoint

Convert the points (x, y) back to original Zone M

?

Convert the coordinates of Zone *M* into the coordinates of Zone 0

Zone 1 → Zone 0



Coordinates in Zone 1: (X, Y) becomes (Y , X) in Zone 0

ConvertToZone0 (X, Y, zone){

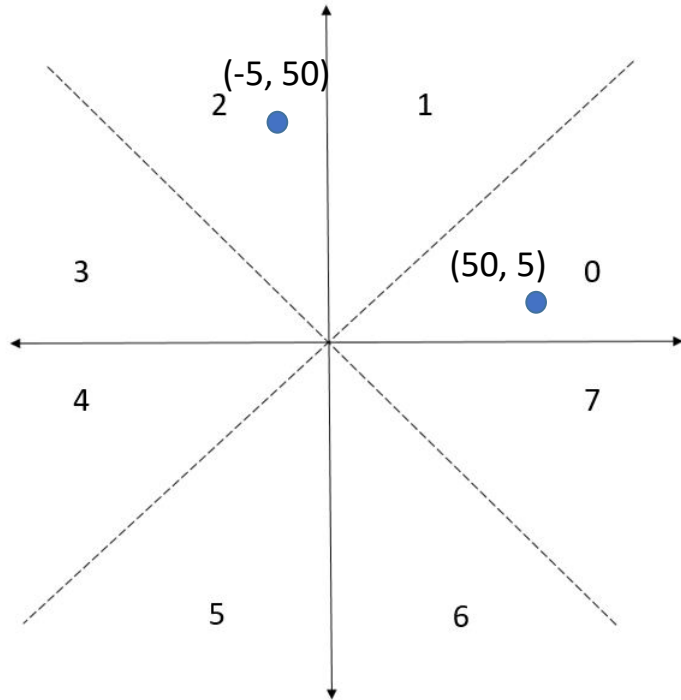
```

    if (zone == 1){
        x = Y, y = X
    }
    return (x, y)
}

```

Convert the coordinates of Zone *M* into the coordinates of Zone 0

Zone 2 $\rightarrow$ Zone 0



Coordinates in Zone 2: (X, Y) becomes $(Y, -X)$ in Zone 0

ConvertToZone0 (X, Y, zone) {

```

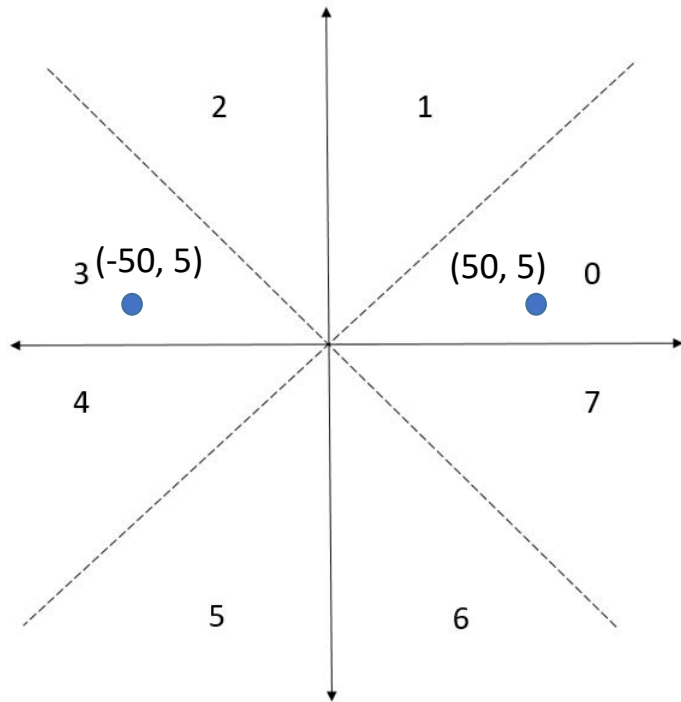
if (zone == 1){
    x = Y, y = X
}
else if (zone == 2){
    x = Y, y = -X
}
return (x, y)

```

}

Convert the coordinates of Zone *M* into the coordinates of Zone 0

Zone 3 → Zone 0



Coordinates in Zone 3: (X, Y) becomes $(-X, Y)$ in Zone 0

ConvertToZone0 (X, Y, zone) {

```

if (zone == 1){
    x = Y, y = X
}
else if (zone == 2){
    x = Y, y = -X
}
else if (zone == 3){
    x = -X, y = Y
}
....
return (x, y)

```

DIY for zone 4, 5, 6, 7

}

```

Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy - dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
    
```

Input (x₁, y₁) to (x₂, y₂) for a line of Zone M, where M = {0, 1, ..., 7}

FindZone

Convert the coordinates of a line in Zone M into the coordinates of a line in Zone 0

ConvertToZone0

Use the existing midpoint line drawing algorithm for Zone 0

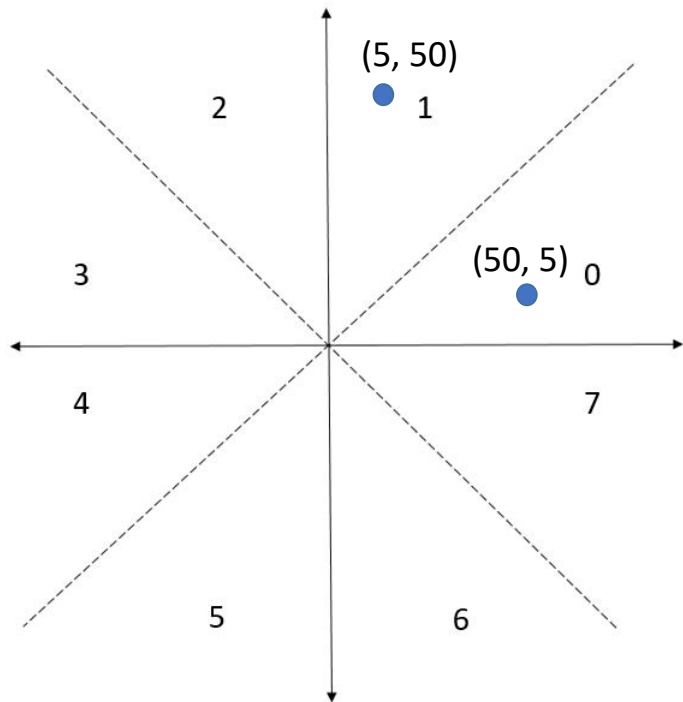
MidPoint

Convert the points (x, y) back to original Zone M

?

Go back to original zone M

Zone 0 $\rightarrow$ Zone 1



Coordinates in Zone 0: (X, Y) becomes (Y , X) in Zone 1

OriginalZone (X, Y, zone){

if (zone == 1){

x = Y, y = X

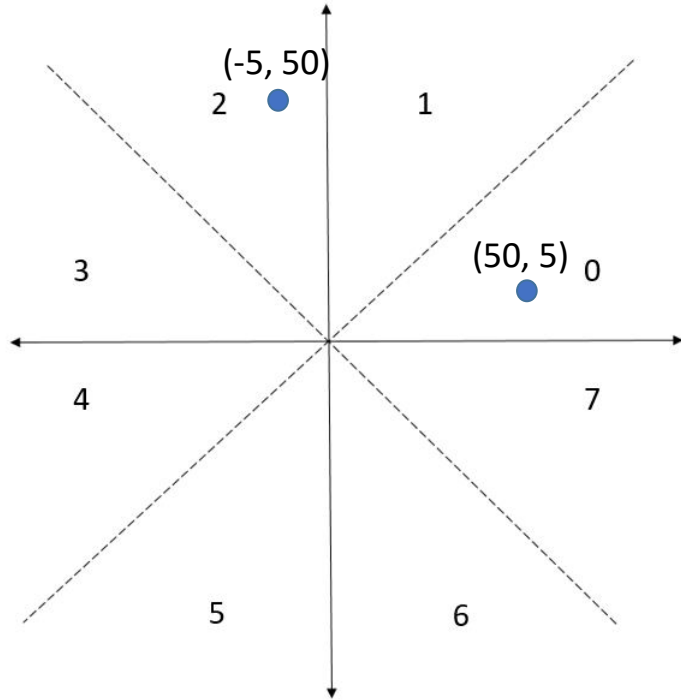
}

return (x, y)

}

Go back to original zone M

Zone 0 $\rightarrow$ Zone 2



Coordinates in Zone 0: (X, Y) becomes (-Y , X) in Zone 2

OriginalZone (X, Y, zone){

```

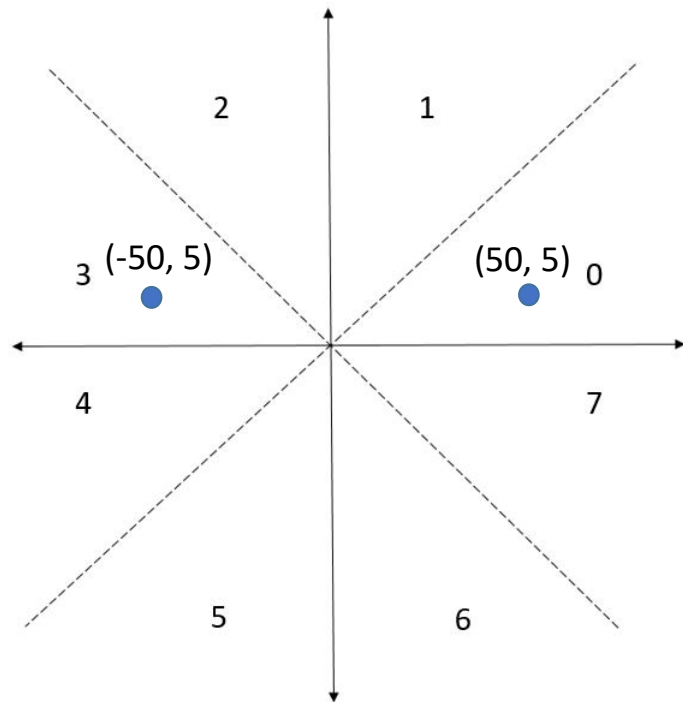
if (zone == 1){
    x = Y, y = X
}
else if(zone == 2){
    x = -Y, y = X
}
return (x, y)

```

}

Go back to original zone M

Zone 0 $\rightarrow$ Zone 3



Coordinates in Zone 0: (X, Y) becomes $(-X, Y)$ in Zone 3

OriginalZone $(X, Y, \text{zone})\{$

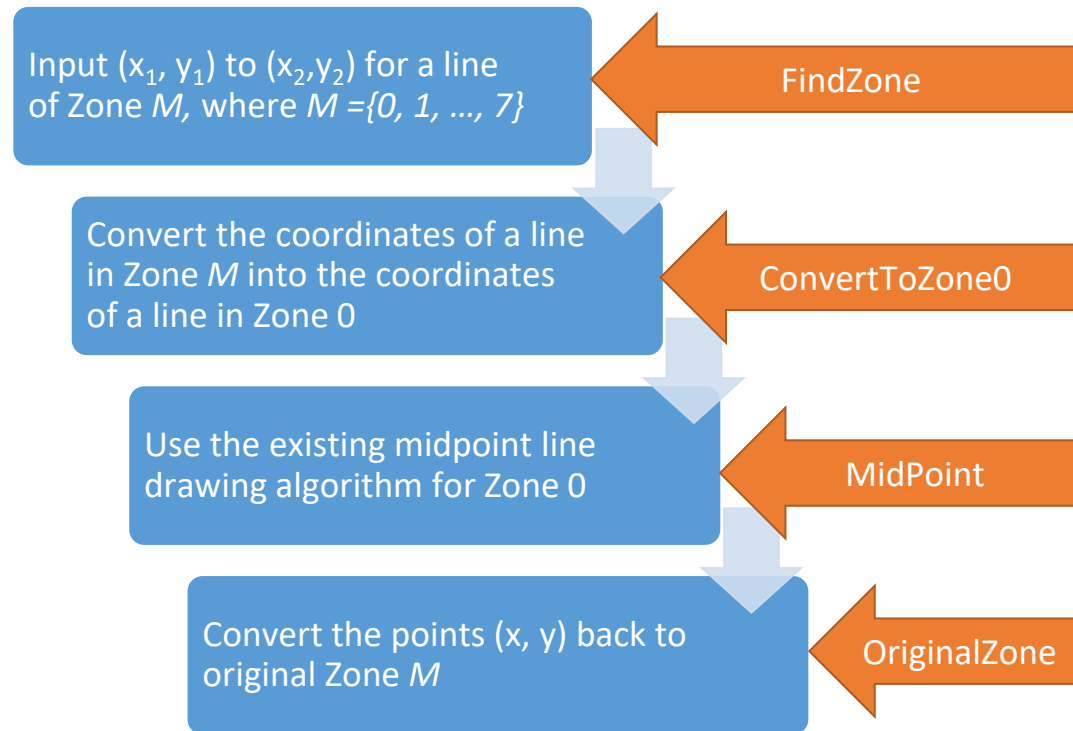
```

if (zone == 1){
    x = Y, y = X
}
else if(zone == 2){
    x = -Y, y = X
}
else if (zone == 3){
    x = -X, y = Y
}
....
return (x, y)

```

DIY for zone 4, 5, 6, 7

}



```

Midpoint (x1, y1, x2, y2){
    dx = x2 - x1; dy = y2 - y1;
    D = 2*dy - dx; ΔNE = 2*(dy-dx); ΔE = 2*dy;
    x = x1; y = y1;
    while( x ≤ x2){
        Draw(x, y);
        x++;
        if (D > 0){
            y++;
            D = D + ΔNE;
        }
        else{
            D = D + ΔE;
        }
    }
}
  
```

Sample Problem and Step by Step Solution

**Find first 7 pixels of the line having endpoints
(-10,-20) and (-20, 70)**

Step 1 (Find out the Zone):

$$dx = -20 + 10 = -10 < 0$$

$$dy = 70 + 20 = 90 > 0$$

$$|dy| > |dx|,$$

So, Zone = 2

Step 2 (Convert to Zone 0):

Zone = 2, so, $(x = Y, y = -X)$

Start: $(-10, -20) \rightarrow (-20, 10)$

End: $(-20, 70) \rightarrow (70, 20)$

So, new endpoints in Zone 0: $(-20, 10)$ and $(70, 20)$

```
ConvertToZone0 (X, Y, zone){
```

```
    if (zone == 1){
```

```
        x = Y, y = X
```

```
    }
```

```
    else if (zone == 2){
```

```
        x = Y, y = -X
```

```
    }
```

```
    else if (zone == 3){
```

```
        x = -X, y = Y
```

```
    }
```

```
    ....
```

```
    return (x, y)
```

```
}
```

Step 3 (Apply MPL for Zone 0):

$$dx' = 70 + 20 = 90,$$

$$dy' = 20 - 10 = 10,$$

$$D = 2*10 - 90 = -70,$$

$$\Delta NE = 2*(10-90) = -160,$$

$$\Delta E = 2*10 = 20$$

Step 4 (Convert to Original Zone):

Orange Colored X' and Y' are in Zone 0,

Now, Convert back to Original Zone

Here, Original Zone = 2, So, $(x = -Y, y = X)$,

Green Colored Part is the actual Pixels (X, Y)

X'	Y'	D	E/NE	Updating D	Pixels (in Zone 2)	
					$X = -Y'$	$Y = X'$
-20	10	-70	E	-50	-10	-20
-19	10	-50	E	-30	-10	-19
-18	10	-30	E	-10	-10	-18
-17	10	-10	E	10	-10	-17
-16	10	10	NE	-150	-10	-16
-15	11	-150	E	-130	-11	-15
-14	11	-130	E	-110	-11	-14

OriginalZone $(X, Y, zone)$

```

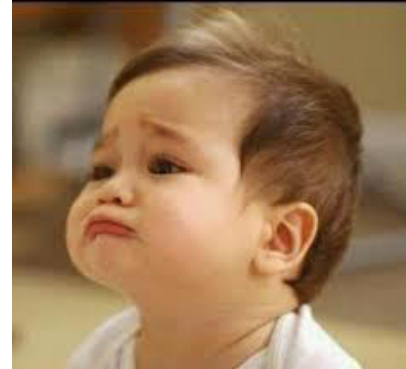
if (zone == 1){
    x = Y, y = X
}
else if(zone == 2){
    x = -Y, y = X
}
else if (zone == 3){
    x = -X, y = Y
}
....
return (x, y)
}

```

So, No More **Staircasing
Effect???**

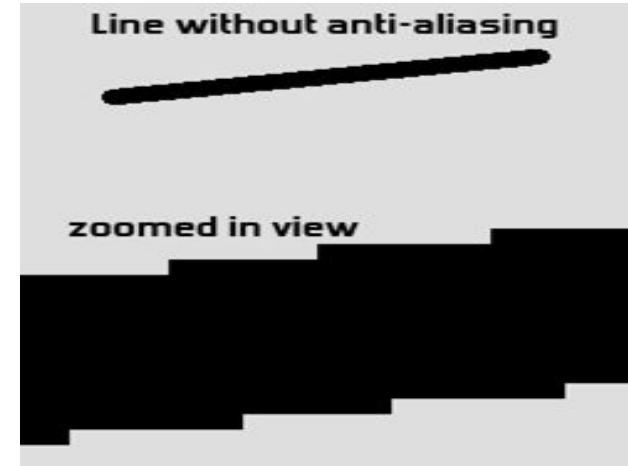


Unfortunately ***NO!!***



Aliasing and Antialiasing

- Aliasing
 - Approximating a continuous entity with discrete samples
 - Jagging / staircasing effect
 - Result of an all-or-nothing approach to scan conversion
 - Each pixel is either colored or left unchanged
- Antialiasing
 - The application of techniques that reduce or eliminate aliasing. Eg. Unweighted Area Sampling



Unweighted Area Sampling

Basic Idea:

- Pixels on the grid overlapped by the rectangle are colored to an appropriate intensity.

General rule : A line contributes to each pixel's intensity an *amount proportional to the percentage* of the pixel's tile it covers.

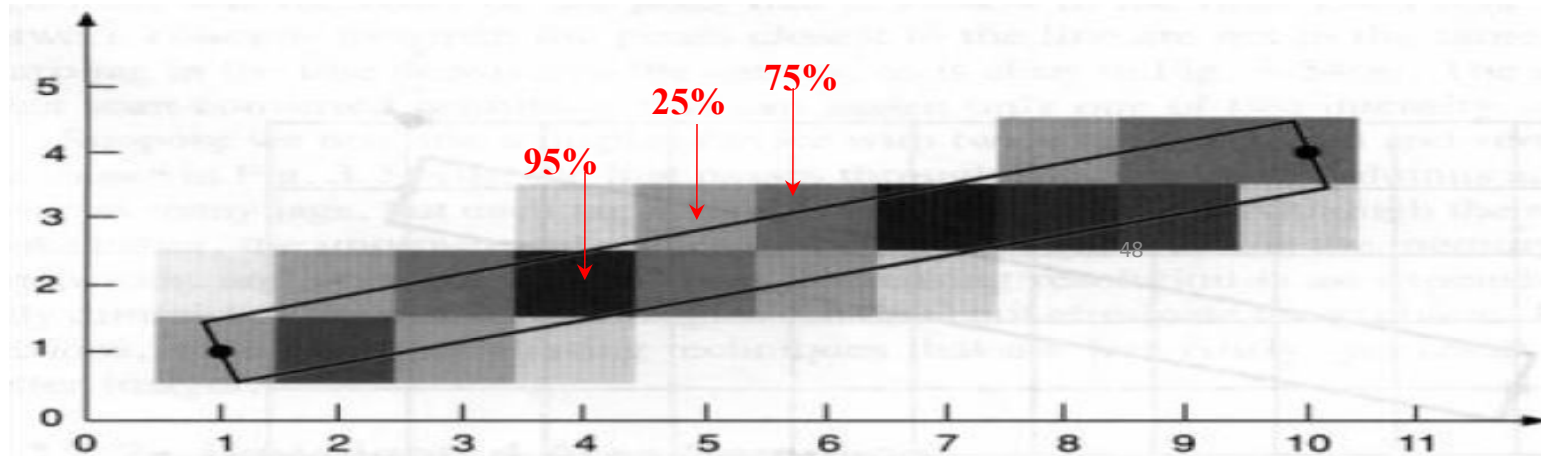


Fig. 3.56 Intensity proportional to area covered.

Reference

Computer Graphics: Principles and Practice: John F. Hughes, James D. Foley, Andries van Dam, Steven K. Feiner (2nd Edition)

Chapter: 3.2